1. INTRODUÇÃO

O crescimento da prática esportiva em ambientes urbanos tem aumentado a demanda por espaços organizados para a realização de atividades físicas, como quadras esportivas. No entanto, muitos desses locais ainda utilizam métodos manuais para controle de reservas, o que pode gerar conflitos de horários, falta de organização e perda de informações.

Diante desse cenário, torna-se relevante o desenvolvimento de sistemas computacionais que automatizem o processo de gerenciamento de reservas, proporcionando maior controle, eficiência e confiabilidade.

Este trabalho tem como objetivo o desenvolvimento de um sistema computacional em linguagem Java, baseado no paradigma de programação orientada a objetos, capaz de gerenciar reservas de quadras esportivas. O sistema permitirá o cadastro de quadras, clientes e reservas, além da realização de consultas, listagens e controle de horários.

A persistência dos dados será realizada por meio de arquivos no formato CSV, garantindo simplicidade na implementação e eficiência no armazenamento.

2. REFERENCIAL TEÓRICO

2.1 Programação Orientada a Objetos

A programação orientada a objetos (POO) é um paradigma que organiza o software em classes e objetos, permitindo maior reutilização de código e melhor organização.

Principais conceitos aplicados:

- Encapsulamento (proteção de dados)
- Abstração (modelagem do mundo real)
- Herança (possível expansão futura)
- Polimorfismo (flexibilidade de métodos)

2.2 Modelagem de Sistemas

A modelagem é essencial para definir a estrutura do sistema antes da implementação. Neste projeto, será utilizada a UML para representar as classes e seus relacionamentos.

2.3 Estruturas de Dados

Será utilizada a estrutura:

- **ArrayList**
    - Para armazenar listas de quadras, clientes e reservas

2.4 Persistência de Dados

A persistência será feita via arquivos CSV, permitindo:

- Armazenamento simples
- Leitura e escrita rápida
- Fácil manutenção

2.5 Tratamento de Exceções

O tratamento de erros será realizado utilizando estruturas de exceção do Java, garantindo que o sistema não seja interrompido por entradas inválidas ou falhas de leitura.

3. PROJETO DO SISTEMA

3.1 Tema

Sistema de Reserva de Quadras Esportivas (Quadra Fácil)

3.2 Visão Geral

O sistema permitirá o gerenciamento de:

- Quadras esportivas
- Clientes
- Reservas de horários

3.3 Requisitos Funcionais

O sistema deve permitir:

- Cadastrar quadras
- Cadastrar clientes
- Realizar reservas
- Listar quadras
- Listar reservas
- Buscar reservas por cliente
- Verificar disponibilidade de horários
- Ordenar reservas por data

3.4 Requisitos Não Funcionais

- Interface via terminal
- Persistência em CSV
- Código organizado e modular
- Validação de dados obrigatória
- Tratamento de erros

3.5 Modelagem das Classes

📌 Classe Quadra

Atributos:

- id (int)
- nome (String)
- tipo (String) → futebol, vôlei, etc.
- precoHora (double)

📌 Classe Cliente

Atributos:

- id (int)
- nome (String)
- telefone (String)

📌 Classe Reserva

Atributos:

- id (int)
- clienteId (int)
- quadraId (int)
- data (String)
- hora (String)

3.6 Relacionamentos

- Cliente → Reserva (1:N)
- Quadra → Reserva (1:N)

3.7 Diagrama UML (DESCRIÇÃO)

Quadra

- id
- nome
- tipo
- precoHora

Cliente

- id
- nome
- telefone

Reserva

- id
- clienteId
- quadraId
- data
- hora

3.8 Regras de Negócio

- Não permitir reserva no mesmo horário para a mesma quadra
- Cliente deve existir para realizar reserva
- Quadra deve existir
- Campos obrigatórios

4. DESENVOLVIMENTO

4.1 Arquitetura

O sistema será dividido em:

- Model (dados)
- Service (lógica)
- Util (arquivos)
- Main (interface)

4.2 Estruturas Utilizadas

- ArrayList<Quadra>
- ArrayList<Cliente>
- ArrayList<Reserva>

4.3 Persistência

Arquivos:

- quadras.csv
- clientes.csv
- reservas.csv

4.4 Interface

Menu textual:

1 - Cadastrar Quadra

2 - Listar Quadras

3 - Cadastrar Cliente

4 - Listar Clientes

5 - Realizar Reserva

6 - Listar Reservas

7 - Buscar Reserva por Cliente

8 - Verificar Disponibilidade

0 – Sair

4.5 Validações

- ID único
- Campos obrigatórios
- Não permitir conflito de horário

4.6 Tratamento de Erros

- Entrada inválida
- Arquivo inexistente
- Dados inconsistentes

5. RESULTADOS E DISCUSSÃO

Durante os testes realizados, o sistema demonstrou ser eficiente no controle de reservas, evitando conflitos de horários e garantindo a integridade dos dados.

Os testes incluíram:

- Cadastro de entidades
- Criação de reservas
- Verificação de conflitos
- Busca e listagem

O uso de CSV mostrou-se adequado para o contexto acadêmico.

💻 📦 GUIA COMPLETO PARA O DESENVOLVEDOR

📂 Estrutura do Projeto:

```
src/
 ├── model/
 │   ├── Quadra.java
 │   ├── Cliente.java
 │   └── Reserva.java
 ├── service/
 │   └── SistemaReserva.java
 ├── util/
 │   └── ArquivoUtil.java
 ├── Main.java
```

📦 ESPECIFICAÇÃO DAS CLASSES:

🔷 Quadra.java

- atributos privados
- construtor
- getters/setters
- toString()

🔷 Cliente.java

- id, nome, telefone

🔷 Reserva.java

- id
- clienteId
- quadraId
- data
- hora

🔷 SistemaReserva.java

Responsabilidades:

- Gerenciar listas
- Aplicar regras de negócio

Métodos obrigatórios:

- cadastrarQuadra()
- cadastrarCliente()
- realizarReserva()
- listarQuadras()

- listarClientes()
- listarReservas()
- buscarReservasPorCliente()
- verificarDisponibilidade()

◈ ArquivoUtil.java

Métodos:

- lerQuadras()
- salvarQuadras()
- lerClientes()
- salvarClientes()
- lerReservas()
- salvarReservas()

⚙ REGRAS IMPORTANTES:

📌 Regra de conflito (ESSENCIAL)

Antes de salvar reserva:

Se existir mesma quadra + mesma data + mesma hora → bloquear

📌 Formato CSV

Quadra:

(id,nome,tipo,precoHora)

Cliente:

(id,nome,telefone)

Reserva:

(id,clienteId,quadraId,data,hora)

📌 Persistência

- Carregar tudo ao iniciar
- Salvar ao inserir

📌 Validações

- Cliente existe?
- Quadra existe?
- Horário disponível?

📌 Tratamento de erros

- try/catch obrigatório
- evitar crash

🔁 FLUXO DO SISTEMA:

Início

↓

Carregar arquivos

↓

Menu

↓

Executar opção

↓

Validar dados

↓

Salvar

↓

Voltar ao menu

🧪 TESTES OBRIGATÓRIOS

- Criar reservas conflitantes (deve bloquear)
- Buscar cliente inexistente
- Testar erro de digitação
- Testar arquivos vazios

🔥 DIFERENCIAL PRA SUA NOTA

Se fizer isso, você se destaca:

- Mostrar validação de conflito funcionando
- Código organizado em camadas
- Métodos bem nomeados
- Comentários claros
- Menu intuitivo